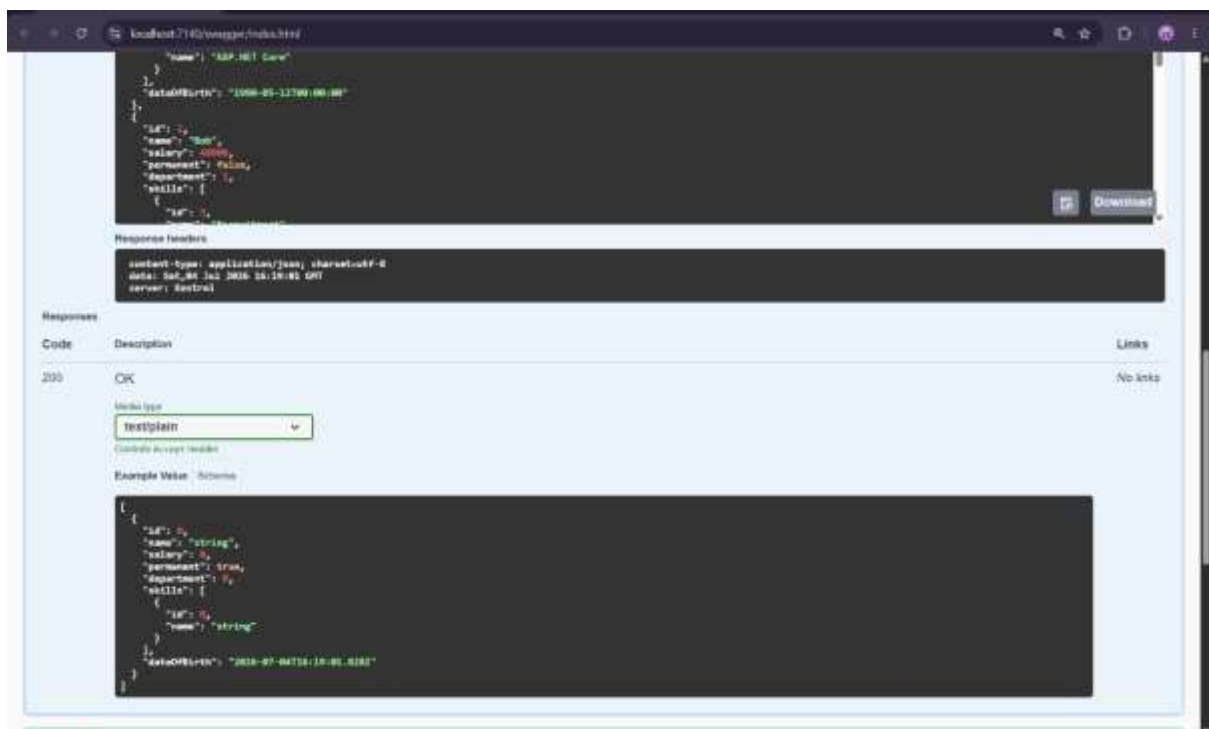
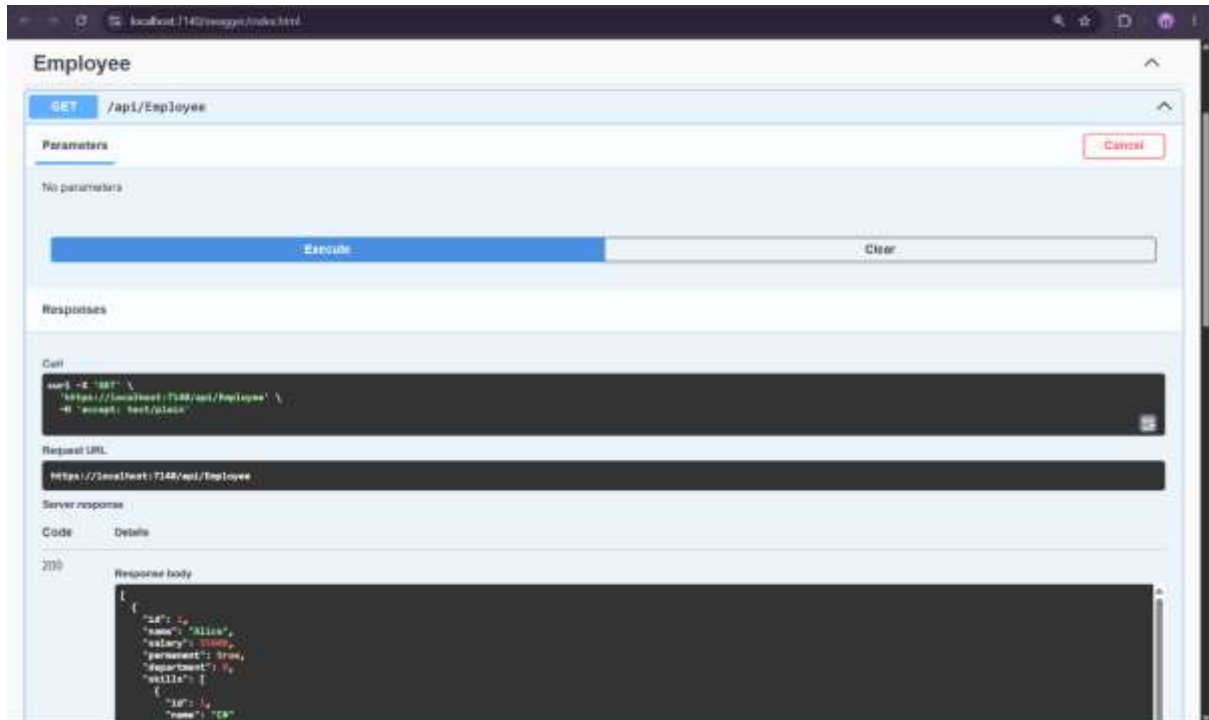


Asp.net Core 8.0 Web API

-Web Api using custom model class :

Output :



Create a Custom action filter for Authorization :

```
using Microsoft.AspNetCore.Mvc;
```

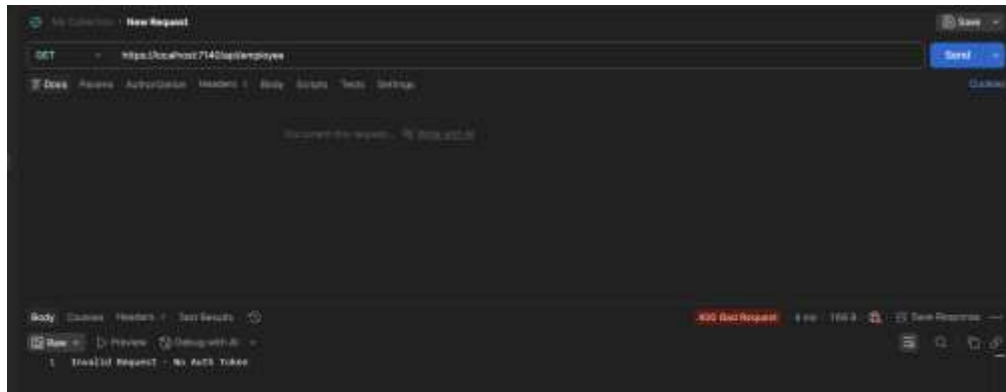
```
using Microsoft.AspNetCore.Mvc.Filters;
```

```
namespace EmployeeApi.Filters
```

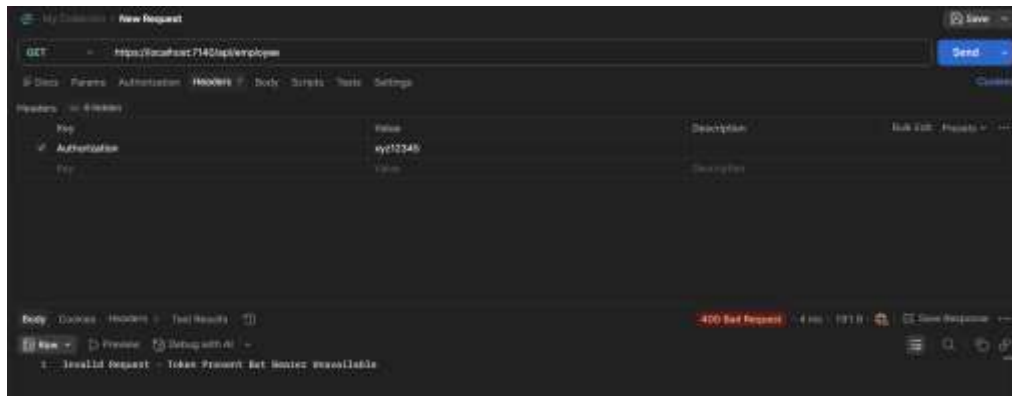
```
{  
    public class CustomAuthFilter : ActionFilterAttribute  
    {  
        public override void OnActionExecuting(ActionExecutingContext context)  
        {  
            if (!context.HttpContext.Request.Headers.ContainsKey("Authorization"))  
            {  
                context.Result = new BadRequestObjectResult("Invalid Request - No Auth  
Token");  
                return;  
            }  
  
            string authHeaderValue =  
context.HttpContext.Request.Headers["Authorization"].ToString();  
  
            if (!authHeaderValue.Contains("Bearer"))  
            {  
                context.Result = new BadRequestObjectResult("Invalid Request - Token Present  
But Bearer Unavailable");  
                return;  
            }  
  
            base.OnActionExecuting(context);  
        }  
    }  
}
```

Output :

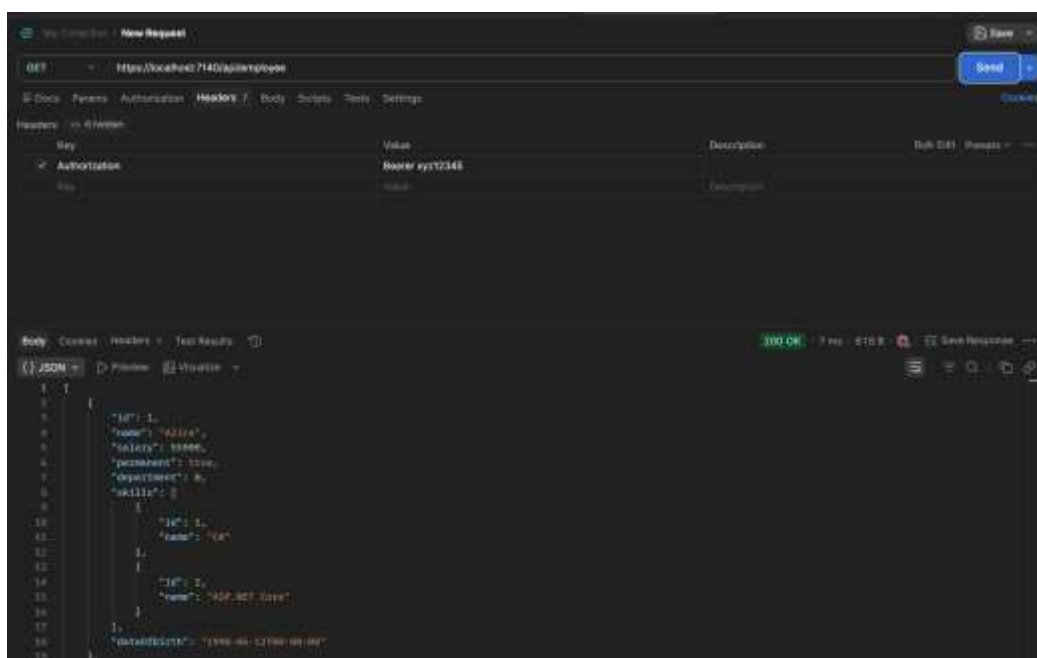
No Authorization header at all



Authorization header present but without "Bearer"



Correct Authorization header



Custom Exception filter :

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace EmployeeApi.Filters
{
    public class CustomExceptionHandler : IExceptionHandler
    {
        public void OnException(ExceptionContext context)
        {
            string exceptionMessage = context.Exception.Message;
            string exceptionStackTrace = context.Exception.StackTrace ?? string.Empty;

            string logContent = $"Time : {DateTime.Now} {Environment.NewLine}" +
                               $"Message : {exceptionMessage} {Environment.NewLine}" +
                               $"StackTrace : {exceptionStackTrace} {Environment.NewLine}" +
                               $"----- {Environment.NewLine}";

            string logFilePath = Path.Combine(AppContext.BaseDirectory, "ExceptionLog.txt");
            File.AppendAllText(logFilePath, logContent);

            context.Result = new ObjectResult(new
            {
                Statuscode = 500,
                Message = "An unexpected error occurred : " + exceptionMessage
            })
            {
                StatusCode = 500
            };
        }
    }
}
```

```
context.ExceptionHandled = true;
    }
}
}
```

